NAME: RAUL DMONTY

ROLL NO: 18 BATCH: A1

LR USING OLS

```
#Step-1 imports
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from scipy.optimize import minimize
```

```
#Step-2 Input data
x = np.array([10,20,30,50]).reshape(-1,1)
y = np.array([12,21,29,48])
```

```
#Step-3 create model and fit data
model = LinearRegression()
model.fit(x,y)
```

```
▾ LinearRegression
LinearRegression()
```

```
#Step-4 get coefficents and print
w = model.coef_[0]
b = model.intercept_
print(f"The slope = {w}, intercept = {b}")
```

```
The slope = 0.8971428571428574, intercept = 2.828571428571422
```

```
#Step-5 make predictions
y_pred = model.predict(x)
print("predictions for training data:")
for xi,yi,ypi in zip(x.flatten(),y,y_pred):
  print(f"x = {xi}, Actual y = {yi}, Predicted y = {ypi}")
```

```
predictions for training data:
x = 10, Actual y = 12, Predicted y = 11.799999999999995
x = 20, Actual y = 21, Predicted y = 20.77142857142857
x = 30, Actual y = 29, Predicted y = 29.742857142857144
x = 50, Actual y = 48, Predicted y = 47.68571428571429
```

```
#Step-6 error calculation
mse = mean_squared_error(y, y_pred)
r2_score = r2_score(y, y_pred)
print("Mean Squared Error:", mse)
print("R^2 Score:", r2_score)
```

```
Mean Squared Error: 0.1857142857142861
R^2 Score: 0.9989463019250253
```

LR USING MLE

```
#Step-1 imports
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from scipy.optimize import minimize
```

```
#Step-2 Input data
x = np.array([10,20,30,50])
y = np.array([12,21,29,48])
```

```
#Step-3 Negayive log likelihood function
def neg_log_likelihood(params):
  w, b = params
  sigma2 = 1 # assume variance = 1
  y_pred = w*x + b
```

```
  nll = 0.5*np.sum((y-y_pred)**2/sigma2)
  return nll
```

```
#initial value
initial_guess = [0,0]
```

```
#Step-4 minimize nll
result = minimize(neg_log_likelihood, initial_guess)
w_mle, b_mle = result.x
print(f"slope: {w_mle}, intercept: {b_mle}")
```

```
slope: 0.8971428246616441, intercept: 2.828572315886469
```

```
#Step-5 predicition
y_pred = w_mle*x + b_mle
print("predictions for training data:")
for xi,yi,ypi in zip(x.flatten(),y,y_pred):
  print(f"x = {xi}, Actual y = {yi}, Predicted y = {ypi}")
```

```
predictions for training data:
x = 10, Actual y = 12, Predicted y = 11.800000562502909
x = 20, Actual y = 21, Predicted y = 20.77142880911935
x = 30, Actual y = 29, Predicted y = 29.74285705573579
x = 50, Actual y = 48, Predicted y = 47.68571354896867
```

```
#Step-6 error calculation
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)
print("Mean Squared Error:", mse)
print("R^2 Score:", r2)
```

```
Mean Squared Error: 0.18571428571451654
R^2 Score: 0.998946301925024
```

LR USING GD (BOTH PARAMETERS)

```
#Step-1 imports
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from scipy.optimize import minimize
```

```
#Step-2 Input data
x = np.array([10,20,30,50])
y = np.array([12,21,29,48])
```

```
#Step-3
w,b = 0,0
alpha = 0.001
n_iter = 10000
n = len(x)
```

```
for i in range(n_iter):
  y_pred = w*x.flatten() + b
  dw = (-2/n)*np.sum(x.flatten()*(y-y_pred))
  db = (-2/n)*np.sum(y-y_pred)
  w -= alpha*dw
  b -= alpha*db
print(f"slope: {w}, intercept: {b}")
```

```
slope: 0.8980343164542518, intercept: 2.7969724110901857
```

LR USING GD (SINGLE PARAMETER)

```
#Step-1 imports
import numpy as np
import matplotlib.pyplot as plt
```

```
#Step-2 Input data
x = np.array([10,20,30,50])
y = np.array([12,21,29,48])
n = len(x)
```

```
#Step-3 Calculate loss function
def loss(w1):
  w0 = np.mean(y) - w1*np.mean(x)
  y_pred = w1 * x + w0
  return np.sum((y - y_pred)**2)
```

```
#Step-4 Calculate gradient of J wrt w1
def gradient(w1):
  w0 = np.mean(y) - w1*np.mean(x)
  y_pred = w1 * x + w0
  return -2/n * np.sum(x * (y - y_pred))
```

```
#Step-5 Gradient Descent
lr = 0.1
w1 = 4
iter = 15
w1_values = []
loss_values = []
for i in range(iter):
    w1_values.append(w1)
    loss_values.append(loss(w1))
    grad = gradient(w1)
    w1 -= lr * grad
```

```
#Step-6 Plot Lost Function and GD
def loss(w):
    return (w - 2)**2
w_space = np.linspace(-2, 4, 200)
loss_space = [loss(w) for w in w_space]
w1_values = [-1, 0, 1, 1.5, 1.8, 1.95, 2.0]
loss_values = [loss(w) for w in w1_values]

plt.figure(figsize=(8, 6))
plt.plot(w_space, loss_space, label="J(w1)")
plt.scatter(w1_values, loss_values, color = 'red', label="GD Steps")
plt.xlabel("w1 (Slope)")
plt.ylabel("Loss (J)")
plt.title("Loss Function and Gradient Descent")
plt.legend()
plt.grid(True)
plt.show()
```